

OOP Java Einführung

Dr.-Ing. Christian Casselt

Redaktionsstand: 23.04.2024

2 x Java

Insel im Südpazifik

Eine der 13000 Inseln von Indonesien

Einwohnerzahl: 120 Millionen

Größe: 50000 km²

Hauptstadt: Jakarta

Java hat einen Vulkan namens Merapi

Exportprodukt: Kaffee (strenger, aromatisch-würziger Geschmack)



Sun Microsystems Inc. einmal über Java:

Java: A simple, object-oriented, distributed, interpreted, robust, secure, architecture neutral, portable, high-performance, multithreaded and dynamic language.



Was ist OOP?

Zitat:

Bei einer Präsentation bei Apple Mitte der 80er-Jahre hielt ein **Mitarbeiter** einen Vortrag über die erste Version der neu entwickelten Programmiersprache Oberon. Diese sollte der nächste große Schritt in der Welt der objektorientierten Sprachen sein.

Da meldete sich **Alan Kay** zu Wort und hakte nach:

»Diese Sprache unterstützt also keine Vererbung?« »Das ist korrekt.«

»Und sie unterstützt keine Polymorphie?« »Das ist korrekt.«

»Und sie unterstützt auch keine Datenkapselung?« »Das ist ebenfalls korrekt.«

»Dann scheint mir das keine objektorientierte Sprache zu sein.«

Der **Vortragende** meinte darauf:

»Nun, wer kann schon genau sagen, was nun objektorientiert ist und was nicht.«

Woraufhin **Alan Kay** zurückgab:

»Ich kann das. Ich bin Alan Kay, und ich habe den Begriff geprägt.«

Quelle: B.Lahres; G.Raýman: Praxisbuch Objektorientierung

object-oriented

Objektorientiert ist ein **Programmierparadigma**
und bedeutet

1. Alles ist ein Objekt
2. Objekte kommunizieren durch das Senden und Empfangen von Nachrichten (welche aus Objekten bestehen)
3. Objekte haben ihren eigenen Speicher (strukturiert als Objekte)
4. Jedes Objekt ist Instanz einer Klasse (welche ein Objekt sein muss)
5. Die Klasse beinhaltet das Verhalten aller ihrer Instanzen (in der Form von Objekten in einer Programmliste)
6. Um eine Programmliste auszuführen, wird die Ausführungskontrolle dem ersten Objekt gegeben und das Verbleibende als dessen Nachricht behandelt

Alan Kay: The Early History of Smalltalk (1993)

Quelle:Wikipedia

Grundkonzepte der OOP

Datenabstraktion

Gemeinsame Eigenschaften und Fähigkeiten eines Gegenstandes oder Begriffs zusammenfassen und beschreiben

Datenkapselung

Zustandsänderung der Objekteigenschaften nur über dafür vorgesehene Operationen (Methoden)

Vererbung

Erweiterung vorhandener Objekte zu neuen Objekten mit zusätzlichen Eigenschaften und Operationen bei bestehenbleibender Kompatibilität zum Ursprungstyp

Polymorphie

Es wird zu Laufzeit entschieden, wie das aktuelle Objekt in Abhängigkeit von der aktuellen Objektreferenz auf eine bestimmte Nachricht reagiert

Inhalt der Vorlesung:

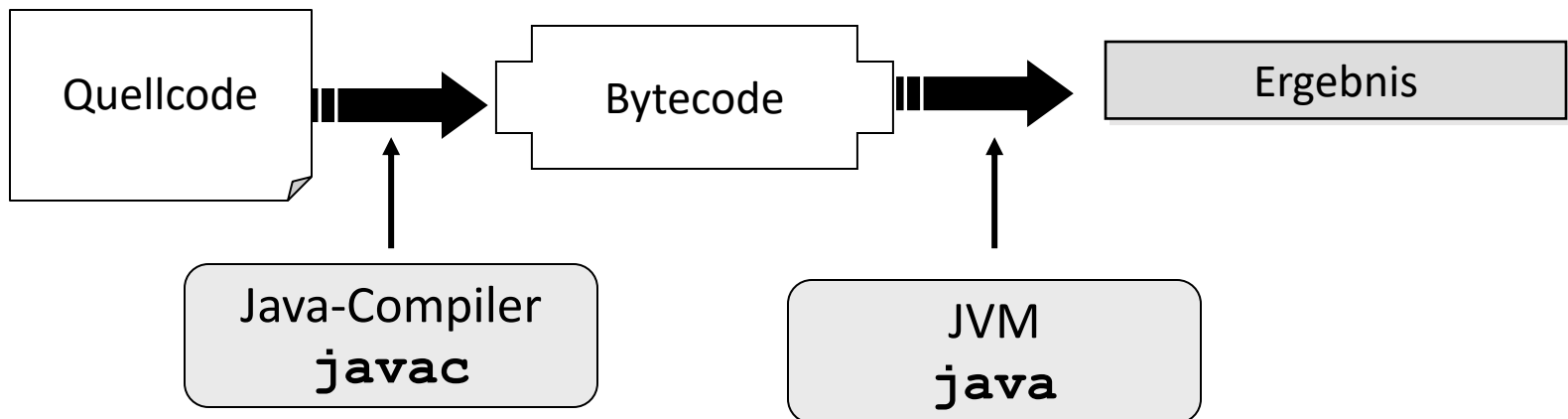
Wie werden diese Grundkonzepte in der Sprache JAVA umgesetzt

interpreted

Der Quellcode wird in von einem Compiler (**javac**) in einen Byte-Code compiliert und dieser wird von einer virtuellen Maschine (**java**) der Zielplattform interpretiert und ausgeführt.

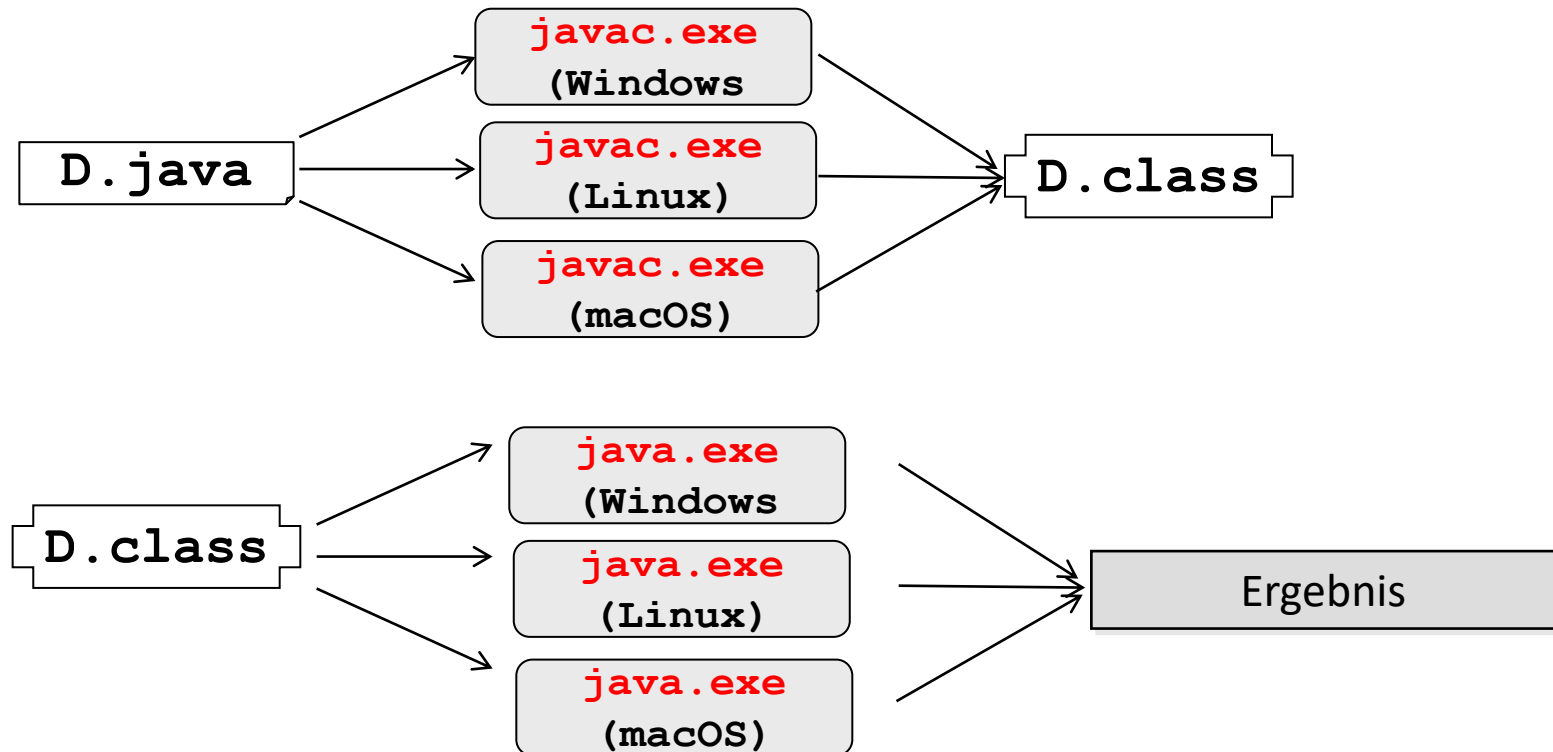
Nachteile: langsamer als die direkte Ausführung von Maschinencode

Vorteile: Bytecode ist plattformunabhängig, gleicher Bytecode wird von der JVM der jeweiligen Plattform ausgeführt



architecture neutral & portable

Die *JVM* ist auf sehr vielen Plattformen implementiert, Unix, MacOS, Windows sowie verschiedenen Browser. Die Programme werden (zur Laufzeit) automatisch an das *Look-and-Feel* der einzelnen Plattformen angepasst.



Der ByteCode einer Klasse

ByteCode der Klasse Beispiell

00000000:	ca	fe	ba	be	00	00	00	31	00	1d	0a	00	06	00	0f	09	Ëp°¼...1.....
00000010:	00	10	00	11	08	00	12	0a	00	13	00	14	07	00	15	07	
00000020:	00	16	01	00	06	3c	69	6e	69	74	3e	01	00	03	28	29	<init>...()
00000030:	56	01	00	04	43	6f	64	65	01	00	0f	4c	69	6e	65	4e	V...Code...LineN
00000040:	75	6d	62	65	72	54	61	62	6c	65	01	00	04	6d	61	69	umberTable...mai
00000050:	6e	01	00	16	28	5b	4c	6a	61	76	61	2f	6c	61	6e	67	n...([Ljava/lang
00000060:	2f	53	74	72	69	6e	67	3b	29	56	01	00	0a	53	6f	75	/String;)V...Sou
00000070:	72	63	65	46	69	6c	65	01	00	0e	42	65	69	73	70	69	rceFile...Beispi
00000080:	65	6c	31	2e	6a	61	76	61	0c	00	07	00	08	07	00	17	ell.java.....
00000090:	0c	00	18	00	19	01	00	11	4a	61	76	61	20	69	73	74	Java ist
000000a0:	20	73	75	70	65	72	21	21	21	07	00	1a	0c	00	1b	00	super!!!.....
000000b0:	1c	01	00	09	42	65	69	73	70	69	65	6c	31	01	00	10	Beispiell...
000000c0:	6a	61	76	61	2f	6c	61	6e	67	2f	4f	62	6a	65	63	74	java/lang/Object
000000d0:	01	00	10	6a	61	76	61	2f	6c	61	6e	67	2f	53	79	73	...java/lang/Sys
000000e0:	74	65	6d	01	00	03	6f	75	74	01	00	15	4c	6a	61	76	tem...out...Ljav
000000f0:	61	2f	69	6f	2f	50	72	69	6e	74	53	74	72	65	61	6d	a/io/PrintStream
00000100:	3b	01	00	13	6a	61	76	61	2f	69	6f	2f	50	72	69	6e	;...java/io/Prin
00000110:	74	53	74	72	65	61	6d	01	00	07	70	72	69	6e	74	6c	tStream...printl
00000120:	6e	01	00	15	28	4c	6a	61	76	61	2f	6c	61	6e	67	2f	n...([Ljava/lang/
00000130:	53	74	72	69	6e	67	3b	29	56	00	21	00	05	00	06	00	String;)V.!.....
00000140:	00	00	00	00	02	00	01	00	07	00	08	00	01	00	09	00	
00000150:	00	00	1d	00	01	00	01	00	00	00	05	2a	b7	00	01	b1	*...±
00000160:	00	00	00	01	00	0a	00	00	00	06	00	01	00	00	00	01	
00000170:	00	09	00	0b	00	0c	00	01	00	09	00	00	00	25	00	02	%..
00000180:	00	01	00	00	00	09	b2	00	02	12	03	b6	00	04	b1	00	².....¶...±.
00000190:	00	00	01	00	0a	00	00	00	0a	00	02	00	00	00	05	00	
000001a0:	08	00	06	00	01	00	0d	00	00	00	02	00	0e				

simple - dynamic - distributed

Wenige Sprachkonstrukte - wenige Schlüsselworte

Keine Pointer - Pointer arbeiten im Hintergrund

Garbage collection – automatische Entsorgung nicht mehr referenzierter Objekte

Keine Mehrfachvererbung

Die Interpreterstruktur von Java ermöglicht das *dynamische Nachladen* von *.class*-files. Es werden beim Programmstart nicht alle Klassen des Programmes geladen, nachladen erfolgt nach Bedarf.

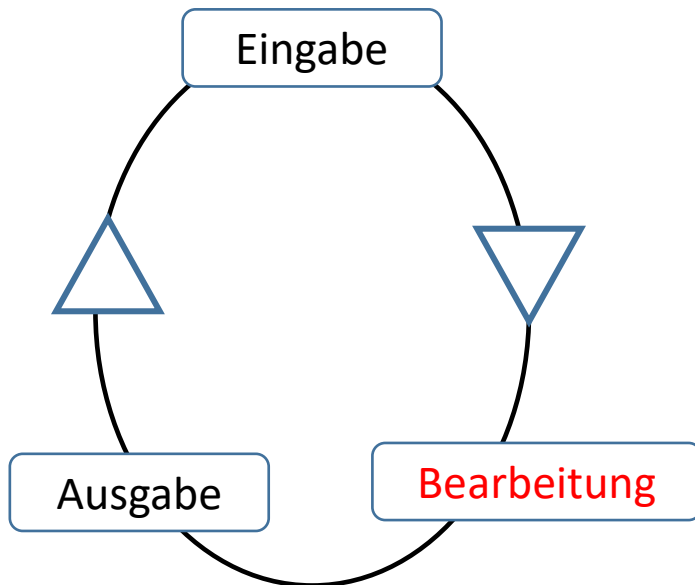
Programme erhalten dadurch eine sehr hohe Flexibilität.

Gut geschriebene (!) Programme können dynamisch an verändernde Bedürfnisse angepasst werden.

dynamic

Nehmen wir an, es läuft aktuell ein Projekt, welches in einer Endlosschleife Daten ausliest und diese aufbereitet ausgibt, z.B.

- für eine ganze Zahl die Fakultät (**es werden mehr Stellen benötigt**)
- für einen Algorithmus eine Festkommazahl (**man braucht einen besseren Algorithmus**)
- **oder eine Sortierung der Daten muss effektiver erfolgen**
- **oder es wird eine weitere Aufbereitung der Daten benötigt.**



Java bietet die Möglichkeit, während der Laufzeit unter Beachtung bestimmter Bedingungen neue Klassen einzubinden und diese auszuführen

Dazu später Beispiele

Einführende Beispiele

1. Programmieren

Quellcode erzeugen und speichern unter **Bsp.java**

2. Kompilieren mittels

`"%JAVA_HOME%\bin\javac" Bsp.java`

Ergebnis: **Bsp.class**

3. Ausführen

Applikation mit

`"%JAVA_HOME%\bin\java" Bsp`

ausführen

Was brauchen wir: **Ein JDK und einen Editor**

Es ist zu empfehlen, mit der Umgebungsvariablen **JAVA_HOME** zu arbeiten, welche ggf. mit dem Pfad zum **aktuellen JDK** gesetzt werden muß!

z.Bsp. `set JAVA_HOME = "C:\Program Files\Java\jdk1.8.0_121"`

Java-Applikation erstellen/ausführen

1. Quellcode schreiben

Wir brauchen eine Klasse mit der Methode

`public static void main(String[] args)` zum Einstieg in die JVM

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Java ist super!!!");  
    }  
}
```

2. Quellcode compilieren

Bytecode erzeugen

`"%JAVA_HOME%\bin\javac" Main.java`

Ergebnis: `Main.class`

3. Bytecode ausführen

JVM interpretiert

`"%JAVA_HOME%\bin\java" Main`

Ergebnis auf der Konsole: `Java ist super`

```
#include <iostream>  
using namespace std;  
int main() {  
    cout << "Hallo User"  
        << endl;  
    cin.get();  
    return 0;  
}
```

Eine Klasse (UML-Javacode)

In der OOP schreibt man Klassen

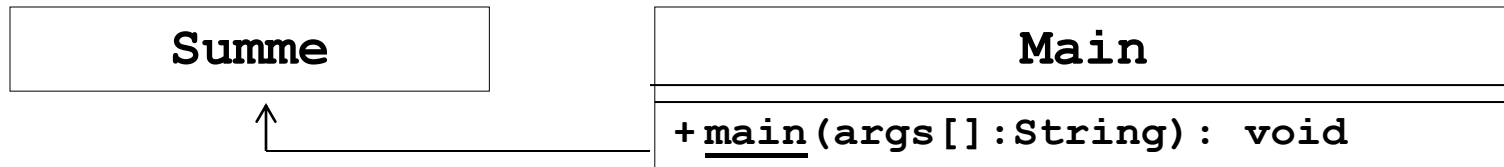
Summe
- n: long
+ setN(n:int): void + summe(): long

//Diese Klasse berechnet die Summe

//der ganzen Zahlen von 1 bis n

```
public class Summe {  
    private long n;  
    public void setN(int x) {  
        n = x;  
    }  
    public long summe() {  
        return n*(n+1)/2;  
    }  
}
```

Eine Klasse in einer Applikation



```
public class Main {
    public static void main(String[] args) {
        Summe summe = new Summe();
        summe.setN(100);
        System.out.println(summe.summe());
    }
}
```

```
class Summe {
public:
    int n;
    void setN(int nn){
        n=nn;
    };
    int summe()
        return n*(n+1)/2;
    };
};
```

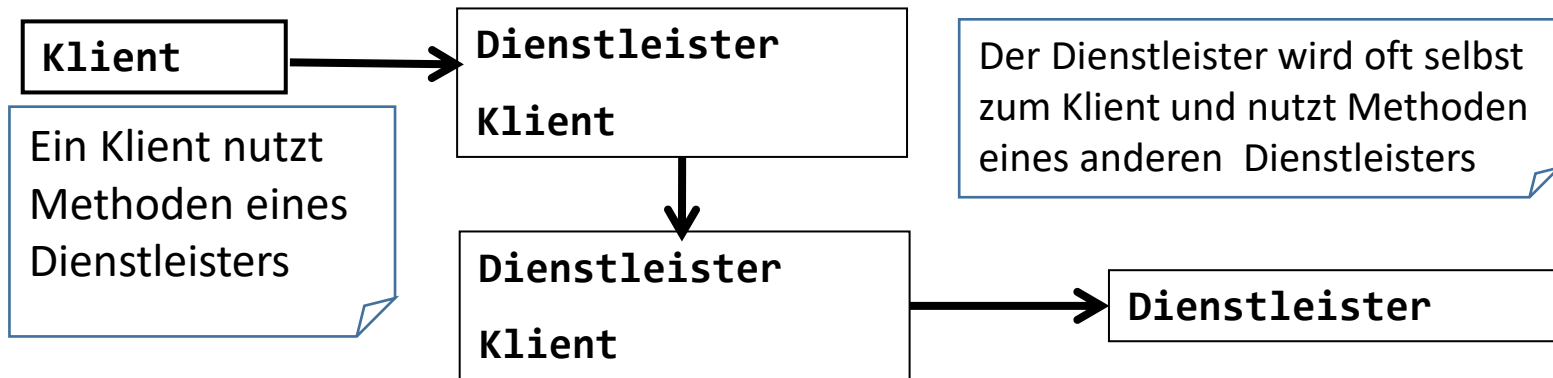
Summe.h

```
#include <iostream>
#include "Summe.h"
using namespace std;
int main() {
    Summe sum;
    sum.setN(12);
    cout << sum.summe() << endl;
    cin.get();
    return 0;
}
```

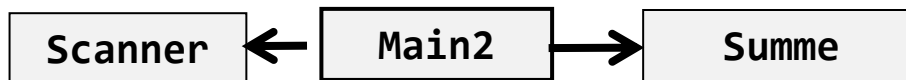
MainS.cpp

Typisch OOP und typisch für Java

Jede Klasse bietet Dienste (Methoden) an
Jedes Objekt kann diese Dienste auf sich selbst ausführen oder andere Objekte mit der Ausführung von Diensten beauftragen

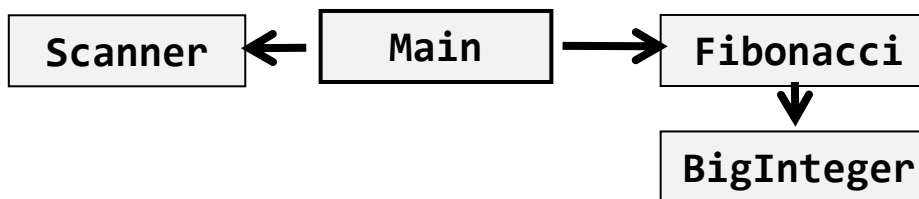


Beispiel2



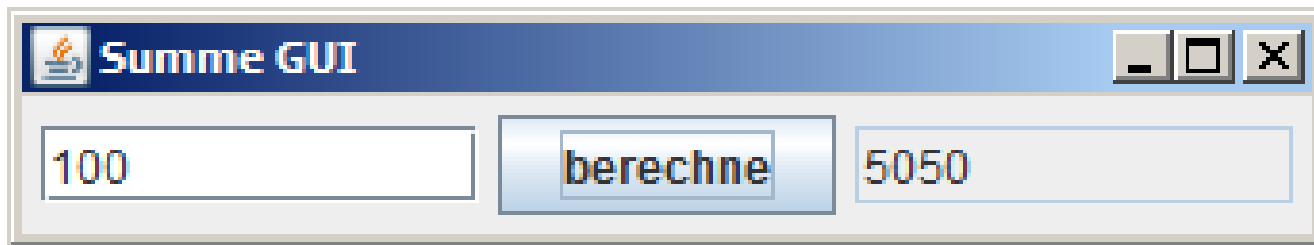
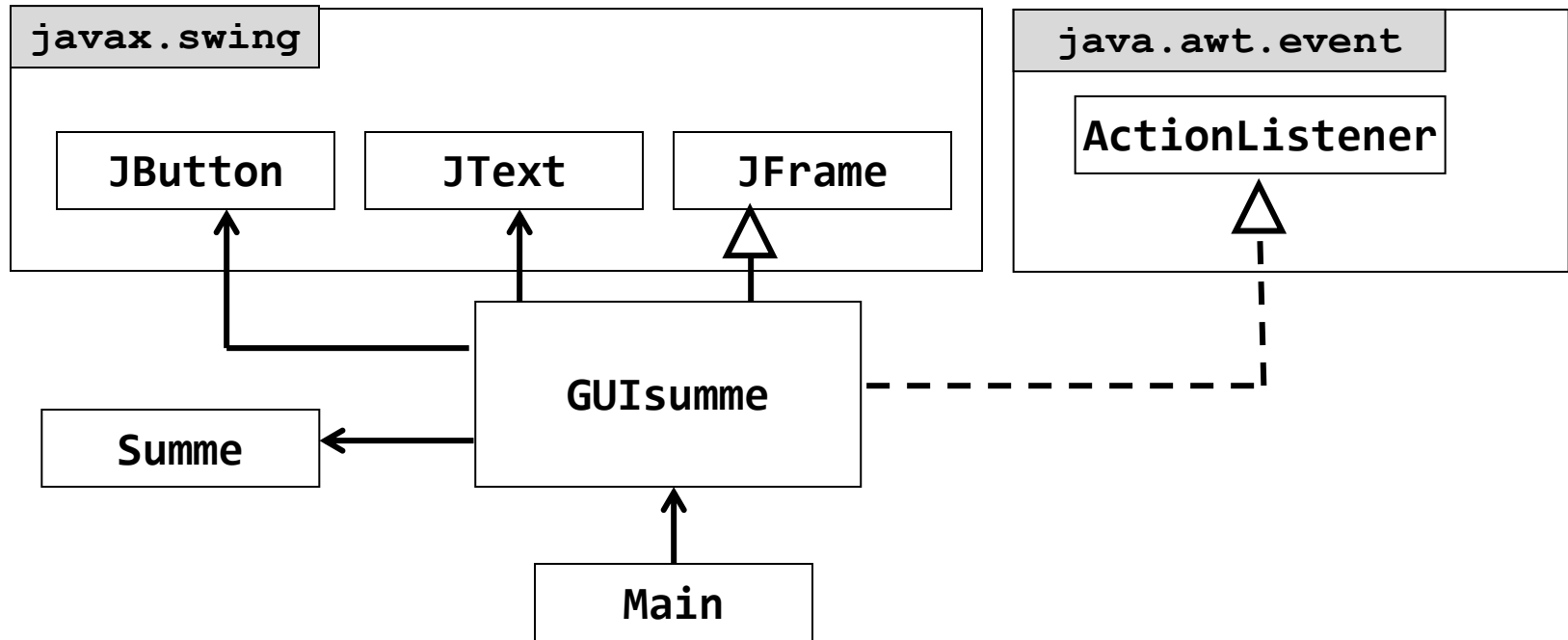
Scanner: Lesen von der Konsole
Main: Hauptklasse für den Start
Summe: Realisieren des Algorithmus

BspFibonacci



Scanner: Lesen von der Konsole
Main: Hauptklasse für den Start
Fibonacci: Realisieren des Algorithmus
BigInteger: Hilfsklasse für große Zahlen

Java-Projekt (BeispielGUI)



Vorgesehene Themen und Probleme

Elementare Sprachelemente

(Bezeichner / Datentypen / Variable / Methoden / Kontrollstrukturen)

Klassen im Detail

(Klassen / Packages / Zugriffsmodifier / Member / static Member / Konstruktoren / Kapselung)

Vererbung ABC Interfaces Polymorphie

(Regeln / Überschreiben / Konstruktoren / Liskovsche Substitution / Schnittstellenkonzept / Klasse Object)

Ausgewählte Probleme und Klassen des JDK

(Delegation vs Vererbung / Beziehungen zwischen Klassen / Generics / ausgew. Klassen: String, Enums, Array, Wrapperklassen / Exceptions / Serialisierung / jar-Files)

JCF (Gleichheit und Sortieren von Objekten / ausgewählte Collections des JCF)

GUI (Grafische UserInterfaces mit SWING)

Es sei darauf hingewiesen, daß die Codebeispiele in der Vorlesung nicht bis in das letzte Detail ausprogrammiert sind, es sollen stets nur wesentliche Aspekte der behandelten Problematik dargestellt werden

Literatur

Nur eine (ganz) kleine empfehlenswerte Auswahl

Ullenboom, Christian: *Java ist auch eine Insel*,
Galileo Computing, 20??, ?.Auflage

Esser, Friedrich: *Java 2, Designmuster und Zertifizierungswissen*
Galileo Computing, 20??

Beide unter <http://www.galileocomputing.de/openbook>

Krüger, G.: *Java 2*
Addison Wesley

API: java 2 Platform SE 6.0 API Spezifikation
<http://java.sun.com/j2se/1.5.0/docs/api/>

The Java Tutorials (SUN)
<http://java.sun.com/docs/books/tutorial>

Bernhard Baltes-Götz & Johannes Götz
Einführung in das Programmieren mit Java 17
Java17.pdf (*liegt bei OPAL*)